SIMATS ENGINEERING

ASSESSMENT TOOL 2 - MEDIUM (Scenario Based Assessment)

COURSE CODE: CSA0904 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO2: *Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks: 50**

ANNEXURE A

Scenario Based Assessment

Code of Conduct:

I Monesh S / 192324027 *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Monesh S

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANNEXURE

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

(a) Enumerate three Collection interfaces suitable for this system.

(b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.

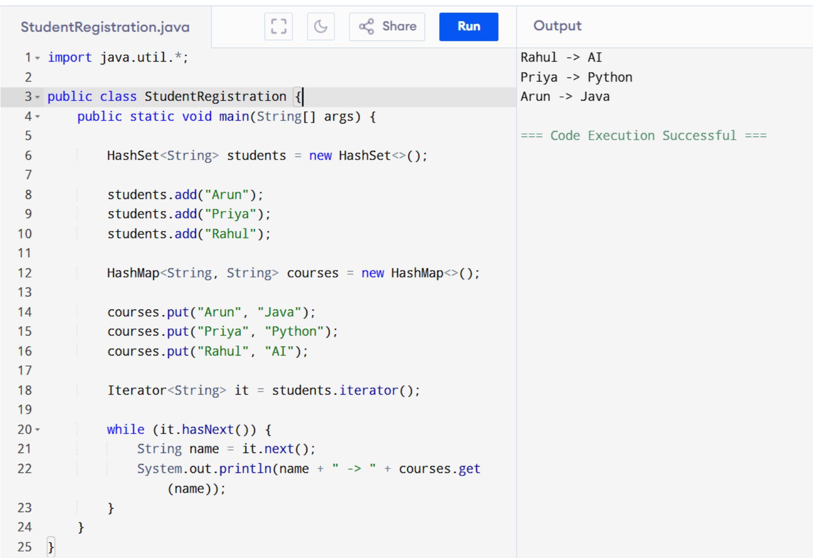
(c) Write a Java program using HashSet and HashMap with Generics to:

- Add students
- Map students to courses
- Display data using Iterator

ANSWER:

```
import java.util.*;
public class StudentRegistration {
    public static void main(String[] args) {

        HashSet<String> students = new HashSet<>();
        students.add("Arun");
        students.add("Priya");
        students.add("Rahul");

        HashMap<String, String> courses = new HashMap<>();

        courses.put("Arun", "Java");
        courses.put("Priya", "Python");
        courses.put("Rahul", "AI");
        Iterator<String> it = students.iterator();

        while (it.hasNext()) {
            String name = it.next();
            System.out.println(name + " -> " + courses.get(name));
        }
    }
}
```

OUTPUT:



Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

(a) List three Collection classes applicable for cart management.

(b) Represent how cart items are stored using a List-based structure.

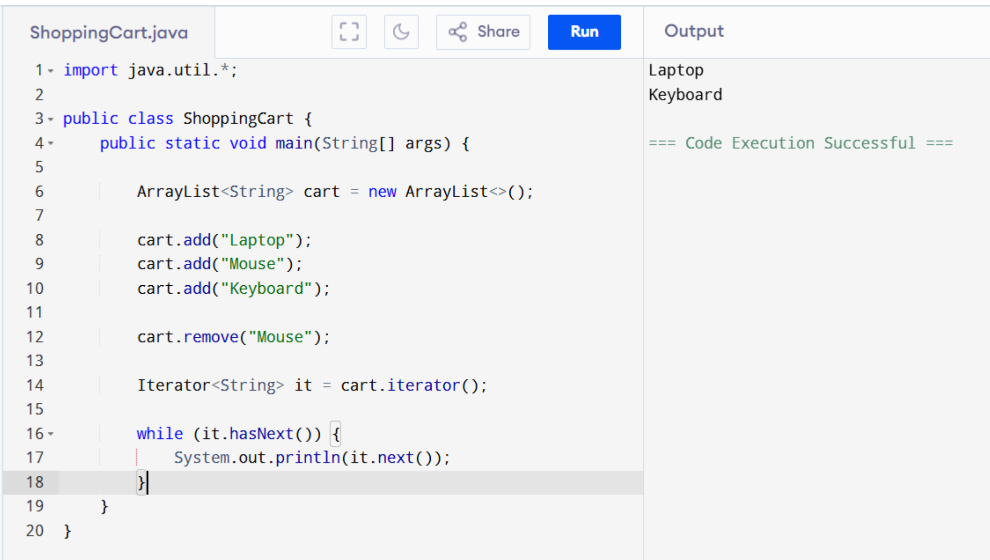
(c) Write a Java program using ArrayList and Iterator to:

- Add items
- Remove an item
- Display all items

ANSWER:

```
import java.util.*;

public class ShoppingCart {
   public static void main(String[] args) {

      ArrayList<String> cart = new ArrayList<>();

      cart.add("Laptop");
      cart.add("Mouse");
      cart.add("Keyboard");

      cart.remove("Mouse");

      Iterator<String> it = cart.iterator();

      while (it.hasNext()) {
         System.out.println(it.next());
      }
   }
}
```

OUTPUT:



Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

(a) Name three Map implementations in Java.

(b) Design a schema-like representation using Map for storing ISBN and book details.

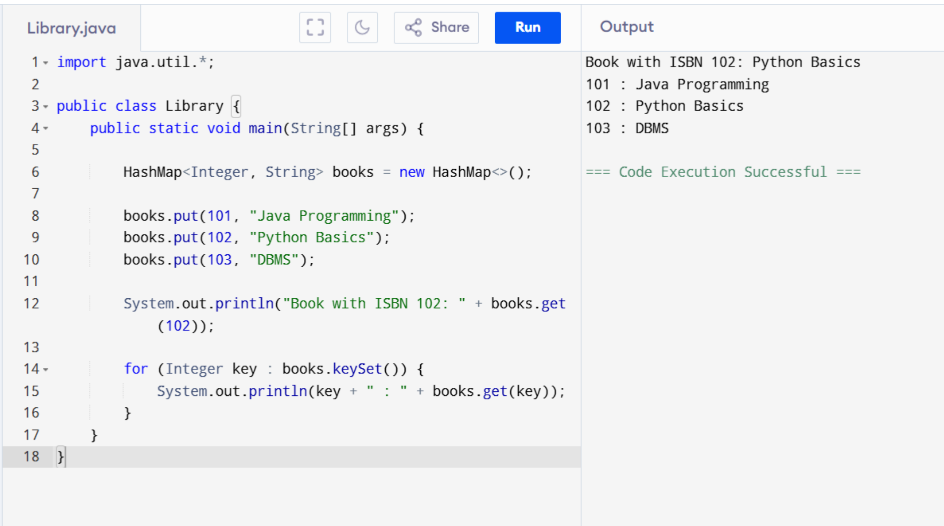
(c) Write a Java program using HashMap to:

- Insert book details
- Retrieve a book
- Display all books

ANSWER:

```
import java.util.*;

public class Library {
    public static void main(String[] args) {

        HashMap<Integer, String> books = new HashMap<>();

        books.put(101, "Java Programming");
        books.put(102, "Python Basics");
        books.put(103, "DBMS");

        System.out.println("Book with ISBN 102: " + books.get(102));

        for (Integer key : books.keySet()) {
            System.out.println(key + " : " + books.get(key));
        }
    }
}
```

OUTPUT:



Q4: Employee Data Processing (10 Marks)

Suppose that a company processes employee records and filters employees based on salary.

(a) List three advantages of Generics in Collections.

(b) Represent how employee data is stored using List with Generics.

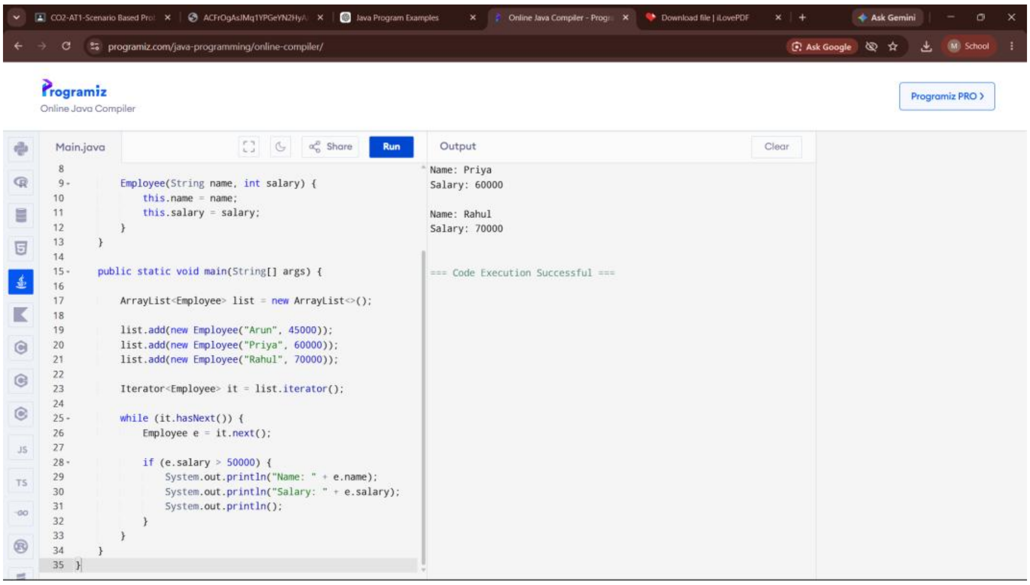
(c) Write a Java program using Iterator to:

- Store employee objects
- Display employees with salary > 50,000

ANSWER:

```
import java.util.*;

public class Main {

    static class Employee {
        String name;
        int salary;

        Employee(String name, int salary) {
            this.name = name;
            this.salary = salary;
        }
    }

    public static void main(String[] args) {

        ArrayList<Employee> list = new ArrayList<>();

        list.add(new Employee("Arun", 45000));
        list.add(new Employee("Priya", 60000));
        list.add(new Employee("Rahul", 70000));

        Iterator<Employee> it = list.iterator();

        while (it.hasNext()) {
            Employee e = it.next();

            if (e.salary > 50000) {
                System.out.println("Name: " + e.name);
                System.out.println("Salary: " + e.salary);
                System.out.println();
            }
        }
    }
}
```

OUTPUT:



Q5: Online Voting System (10 Marks)

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

(a) Identify three suitable Collection types for this system.

(b) Design a structure using Set and Map for voters and vote counting.

(c) Write a Java program using HashSet and HashMap to:

- Record votes
- Prevent duplicate voting • Display results

ANSWER:

```
import java.util.*;

public class VotingSystem {
   public static void main(String[] args) {

      HashSet<String> voters = new HashSet<>();
      HashMap<String, Integer> votes = new HashMap<>();

      String voter = "Arun";

      if (voters.add(voter)) {

         votes.put("A", votes.getOrDefault("A", 0) + 1);

         System.out.println("Vote Recorded");
      } else {
         System.out.println("Already Voted");
      }

      voter = "Arun";

      if (voters.add(voter)) {
         votes.put("B", votes.getOrDefault("B", 0) + 1);
      } else {
         System.out.println("Already Voted");
      }

      System.out.println("Voting Result");

      for (String key : votes.keySet()) {
         System.out.println(key + " : " + votes.get(key));
      }
   }
}
```

OUTPUT:

VotingSystem.java

Share Run

```java
        String voter = "Arun";

        if (voters.add(voter)) {

            votes.put("A", votes.getOrDefault("A", 0) + 1);

            System.out.println("Vote Recorded");
        } else {
            System.out.println("Already Voted");
        }

        voter = "Arun";

        if (voters.add(voter)) {
            votes.put("B", votes.getOrDefault("B", 0) + 1);
        } else {
            System.out.println("Already Voted");
        }

        System.out.println("Voting Result");

        for (String key : votes.keySet()) {
            System.out.println(key + " : " + votes.get(key));
        }
    }
}
```

Output

```
Vote Recorded
Already Voted
Voting Result
A : 1

=== Code Execution Successful ===
```